Exercise

- **Make a simple mini shell**
  - This program runs in an infinite loop until "**exit [status]**" is passed.
    - » If a number is passed to exit (i.e., "`exit 1`"), it should exit with that status.
    - » If a number outside of the range [0, 255] is passed, exit with 255.
    - » If no number is passed, program exits with `EXIT_SUCCESS`.
    - » If status is not a number, print "`exit: STATUS: invalid integer`"
      - » i.e., "`exit: a: invalid integer`"
    - » If multiple arguments are passed (i.e., "`exit 1 a`"),
      - » print "`exit: too many arguments`").
  - The entered command should be executed in a **child process** with execv* functions like execvp.
    - » Executable is in $PATH or current directory.
    - » If the executable does not exist in $PATH or the current directory, print "`COMMAND: command not found`" and the child process should exit with status 127.
  - When the program quits, the parent process must wait for all child processes to terminate before terminating.
  - Error messages should be printed to stderr, not stdout!

Exercise

- **Limitations**
  - Only exec* functions are allowed.
  - Therefore, the following is not allowed.

```
char* cmd;
get_input(&cmd);
system(cmd);
```

  - Passing shell commands to exec* functions is not allowed.
  - Therefore, the following is not allowed.

```
if (pid == 0) {
  execlp("bash", "bash", "-c", cmd, NULL);
  free(cmd);
  fprintf(stderr, "Error: Command not found: %s\n", ???);
  exit(EXIT_FAILURE);
}
```

Exercise

```
./w5
$ ls
Makefile  w5  week5.c  week5.o
$ head -n 7 week5.c
#include <stdio.h>              // printf
#include <stdlib.h>             // free
#include <string.h>             // strtok_r, sprintf
#include <sys/wait.h>           // waitpid
#include <unistd.h>             // execv
#include <editline/readline.h>  // readline
#include <editline/history.h>   // add_history
$ bc -q
1 + 1
2
quit
$ make clean
rm -f week5.o w5
$ ls
Makefile  week5.c
$ make -s
$ something
something: command not found
$ ls
Makefile  w5  week5.c  week5.o
$ bash -c ./w5
$ ./w5
$ exit 1
$ exit
$ exit a
exit: a: invalid integer
$ exit a 1
exit: too many arguments
$ exit 13980925

spl on 🌍 SPL in 2026s/w5/week5 via C v13.3.0-gcc took 11m2s
‼echo $?
255
```

Helper functions

```c
#include <editline/readline.h>  // readline
#include <editline/history.h>   // add_history

enum {
  SUCCESS,
  READLINE_EXIT,
  PARSE_ARGS_ERROR,
  EVALUATE_ERROR
};

cat test

// You have to free the memory allocated by readline
int get_input(char** cmd) {
  if ((*cmd = readline("$ ")) == NULL)
    return READLINE_EXIT;

  if (**cmd == '\0') {
    free(*cmd);
    *cmd = NULL;
  } else add_history(*cmd);
  return SUCCESS;
}
```

Helper functions

```c
#include <stdio.h>                    // fprintf
#include <string.h>                   // strtok_r
#define MAX_ARGS 100

int parse_args(char* cmd, char* args[]) {
  char* save_ptr;
  char* ptr = strtok_r(cmd, " ", &save_ptr);
  int i = 0;

  while (ptr != NULL) {
    if (i >= MAX_ARGS - 1) {
      fprintf(stderr, "Too many arguments\n");
      return PARSE_ARGS_ERROR;
    }
    args[i++] = ptr;
    ptr = strtok_r(NULL, " ", &save_ptr);
  }

  args[i] = NULL;
  return SUCCESS;
}
```

Exercise hint

- **When using libedit,**
  - You have to add `-ledit` at the end when linking in the Makefile
    - » i.e. `gcc (obj file that uses readline).o –ledit`
  - Libedit is installed in Ubuntu by default
    - » But if it is not installed, you can install it with `sudo apt install libedit-dev`
- **A shell operates using REPL (read-eval-print loop)**
  - **R**ead: get the input from user
  - **E**val: evaluate the input
  - **P**rint: print the result
  - **L**oop: Go back to the first step
- **When testing your code, try using programs that take user input (bc, man, python3, etc.)**
- **No restrictions with header files.**

Exercise submission

- **Submit your source code and Makefile**
  - via iCampus
  - Bundle *source code* and *Makefile* with tar command assuming you are in the parent of the week5 directory.

    $ tar caf *STUDENTID_w5.tar.zst week5*
  - We'll grade your submission with **make**
    - » If compilation fails, your points for this exercise will be zero
  - `vim TAR_FILE` should return something like this:

```
" ...
" Browsing tarfile .../2025000000_w5.tar.zst
" ...

week5/
week5/week5.c
week5/Makefile
```